

Full length article

Dynamic reconfiguration in multi-robot agent systems using embedded language models[☆]Shokhikha Amalana Murdivien^a, Jongsu Park^a, Jumyung Um^{a,b,c} *^a Department of Artificial Intelligence, Kyung Hee University, 1732, Deogyong-daero, Yongin-si, 17104, Gyeonggi-do, Republic of Korea^b Department of Industrial and Management Systems Engineering, Kyung Hee University, 1732, Deogyong-daero, Yongin-si, 17104, Gyeonggi-do, Republic of Korea^c Graduate School of XR-AI Convergence, Kyung Hee University, 1732, Deogyong-daero, Yongin-si, 17104, Gyeonggi-do, Republic of Korea

ARTICLE INFO

Keywords:

Autonomous manufacturing
Large Language Models
Multi-robot coordination
Asset administration shell
Retrieval-Augmented Reasoning

ABSTRACT

In modern smart factories, coordinating heterogeneous robotic systems is essential for achieving flexible and scalable automation. This paper presents a communication framework in which each physical asset, such as a robot or camera, operates with an independent Large Language Model (LLM) agent, while a Supervisor Agent performs task interpretation and delegation based on standardized metadata defined in the Asset Administration Shell (AAS). The AAS provides machine-readable descriptions of each asset's skills and operational state, and a retrieval-augmented generation mechanism supplies the Supervisor with task-relevant information during runtime, enabling adaptive task planning and reallocation without extensive reprogramming. The framework was validated through several experiments, including evaluation of locally deployed LLMs for structured task allocation, dynamic delegation in response to AAS updates, inter-agent data transmission, and real-world execution on heterogeneous physical robots. The results demonstrate consistent task execution, reliable message passing, and coordinated behavior across agents. A comparative evaluation further shows that while local LLMs are effective for well-defined tasks, robustness under ambiguous or conflicting instructions depends on the reasoning capability of the underlying model. By combining AAS metadata with distributed LLM agents and a centralized reasoning layer, the approach supports plug-and-play integration of heterogeneous assets while maintaining privacy and enabling offline operation, offering a scalable and interpretable foundation for autonomous manufacturing systems.

1. Introduction

Flexible and scalable production in modern smart factories depends on effective coordination among robots, vision sensors, and control systems. Achieving such coordination requires a shared and standardized representation of industrial assets, including physical equipment and software components. In response to this need, Industry 4.0 has introduced the concept of Asset Administration Shells (AAS) as a standardized digital representation of industrial assets. The AAS mirrors the physical and operational state of industrial assets, providing a structured means for exchanging their current status and capabilities in real time [1,2]. As AAS adoption becomes increasingly standardized across industrial systems, it offers a promising foundation for machine-level understanding and reasoning in collaborative manufacturing environments. With AAS becoming part of standardized initiatives such as those defined in Platform Industrie 4.0 and IEC 63278 [3], its adoption

is expected to grow across both large-scale manufacturers and SMEs (small and medium-sized enterprises).

Previous approaches to collaborative robot systems have relied on static communication protocols and predefined control hierarchies. Communication between robots typically depends on hardcoded interfaces, and coordination is centrally controlled or manually defined. While existing adapters or middleware can unify message formats, they cannot support collaboration across heterogeneous assets without extensive manual engineering, such as implementing vendor-specific coordination logic or hardcoded inter-robot interfaces, which often require modification when system configurations change [1]. Reducing this integration burden requires a coordination mechanism that decouples system-level task reasoning from robot-specific implementations, which is not achievable through static adapters or rule-based mapping. As a result, these systems cannot dynamically respond to changes such

[☆] This article is part of a Special issue entitled: 'AGI4RoboticsManufacturing' published in Robotics and Computer-Integrated Manufacturing.

* Corresponding author at: Department of Industrial and Management Systems Engineering, Kyung Hee University, 1732, Deogyong-daero, Yongin-si, 17104, Gyeonggi-do, Republic of Korea.

E-mail address: jayum@khu.ac.kr (J. Um).

<https://doi.org/10.1016/j.rcim.2026.103308>

Received 31 July 2025; Received in revised form 13 March 2026; Accepted 3 April 2026

Available online 7 April 2026

0736-5845/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

as agent failure or task reordering. Moreover, collaboration across heterogeneous robots is difficult to scale due to the absence of a unified communication layer that supports transparent, language-based interaction without requiring robot-specific coordination logic. Addressing this limitation requires a communication paradigm that is both machine-interpretable and human-understandable, enabling agents to exchange task information through flexible, language-based interaction rather than vendor-specific code.

Large Language Models (LLMs) have recently emerged as a potential solution to this challenge by enabling robots to communicate and coordinate using natural language instructions. Unlike traditional rule-based or learning based interfaces that rely on fixed data mappings, LLMs can flexibly interpret and generate context-dependent responses, allowing agents to interpret high-level task descriptions for coordination purposes without requiring explicit programming of inter-robot interaction logic. Their capacity for reasoning and generalization makes them suitable for supporting coordination in heterogeneous environments where explicit programming of every interaction is impractical. However, current LLM-based robotic applications often depend on encoding all relevant task and agent information within the initial prompt, or alternatively, rely on real-time perception inputs for environmental understanding. In the context of smart factories, where safety, precision, and operational traceability are critical, these methods pose significant risks, since missing or outdated context in the prompt or unreliable perception can lead to critical failures.

In addition, conventional coordination frameworks often rely on rule-based planners or mathematical optimization methods to assign tasks [4]. While these approaches are effective in structured and predictable environments, they require explicit task rules and extensive model engineering. A major limitation is that they are computationally expensive and difficult to reconfigure when the number of robots increases or when system composition changes [5]. Consequently, these rigid methods scale poorly in heterogeneous settings and cannot easily handle dynamic task variations or high-level human instructions without significant manual reprogramming [6]. These challenges highlight the need for a more flexible reasoning mechanism that can interpret high-level intent and adapt to changes during execution.

To overcome these limitations, this paper leverages AAS as a standardized and structured runtime knowledge source, enabling LLM agents to reason over real-time metadata through a Retrieval-Augmented Generation (RAG) mechanism. In the proposed method, real-time refers solely to the AAS providing up-to-date asset information. This approach ensures that task allocation decisions reflect the current status and capabilities of each agent while supporting both autonomous operation and human instruction. By integrating AAS-driven metadata, the framework minimizes the need for additional perception systems or complex integration layers.

This paper proposes a communication framework in which each robot and vision sensor (camera) is paired with an LLM agent that performs task planning and coordination based on the asset's AAS. The system adopts a distributed architecture in which each asset hosts its own LLM-based agent. This design enables plug-and-play integration, as agents operate independently without requiring centralized reconfiguration when assets are added, removed, or replaced. A Supervisor agent interprets high-level user commands and allocates subtasks by dynamically retrieving relevant metadata from the AAS document corpus. Through RAG, the system maintains up-to-date context awareness while preserving flexibility and interpretability. All agents communicate via a language-based messaging protocol, allowing seamless interaction even across heterogeneous platforms.

In this framework, natural language is used exclusively for coordination and task communication, while each physical agent executes actions independently using its own predefined control functions. To ensure practical deployability in industrial environments, the framework is designed to operate with locally deployed LLMs, avoiding reliance on external cloud services and enabling on-premise execution

that better satisfies cost, connectivity, and data privacy requirements, particularly for SMEs.

The main contributions of this work are summarized as follows:

- (1) A distributed multi-agent communication framework is introduced in which each physical asset (robot or camera) is paired with an independent LLM agent. This design enables plug-and-play integration, allowing assets to be added, removed, or substituted without reconfiguring system-level logic.
- (2) AAS is used as a standardized metadata layer describing each asset's skills, operational state, and configuration in a vendor-neutral manner. By representing capabilities as natural-language skills, the framework supports coordination across heterogeneous robots without relying on vendor-specific APIs.
- (3) A retrieval-based reasoning mechanism is integrated to allow the Supervisor Agent to query only task-relevant AAS entries before each decision step. This enables dynamic task planning and reallocation based on updated asset status, without hardcoded rules or static workflow assumptions.
- (4) A natural-language messaging protocol is established for communication among agents, supporting flexible task interpretation, adaptive coordination, and transparent interaction in the system.
- (5) The full framework is implemented and validated on physical robots with different platforms and capabilities, demonstrating successful task execution, dynamic reallocation, and reliable inter-agent coordination in real-world settings.

The remainder of this paper is organized as follows: Section 2 provides an overview of existing literature relevant to LLM-based robotic coordination and standardized asset representation methods. Section 3 describes the architecture and AAS submodel for LLM LLM-based agent system. Section 4 presents experimental validation results, demonstrating performances of local deployed LLM, dynamic allocation, and communication accuracy. Finally, Sections 5 and 6 discuss findings and conclusions drawn from this research.

2. Literature survey

2.1. Industry 4.0 and smart factories

Smart manufacturing has achieved significant advancements through the adoption of big data and artificial intelligence technologies. However, several challenges remain, including variability in production demands, the need for dynamic task reassignment, and the goal of reducing human intervention. Traditional manufacturing methods therefore need to be revised to meet the goals of Industry 4.0, such as sustainability and responsiveness [7].

Interoperability among heterogeneous assets is a key requirement to achieve these goals. In modern factories, different robots and sensors may perform similar actions but express them using different command structures or parameter formats, which hinders direct information exchange and highlights the need for a common metadata structure [8, 9].

Several approaches have been used to represent and exchange information in industrial and robotic systems, including OPC-UA based information models and ROS-based communication frameworks [10]. OPC-UA provides standardized interfaces for industrial communication but is primarily designed for deterministic data exchange and hierarchical control [11]. ROS enables efficient message passing among robots but relies on tightly coupled system configurations [12], which makes cross-vendor interoperability and asset-level representation difficult [13,14].

Approaches such as AML, OPC UA, and ROS typically assume statically engineered configurations. Manufacturing systems are commonly preconfigured during the engineering phase, making runtime reconfiguration time-consuming in dynamic environments [15]. AML is mainly

used during system design and is not intended for structural device changes at runtime [11]. Similarly, OPC-UA information models require manual adaptation when system structures change, and maintaining consistent models across system layers remains challenging [16,17]. In addition, the semantic capabilities of AML and OPC-UA are limited, which can complicate the automatic interpretation of equipment capabilities in dynamic manufacturing scenarios [18–20]. Consequently, task allocation and coordination logic in such systems are often predefined or hardcoded. While this deterministic design enables efficient and low-latency execution, it limits the ability of the system to adapt to runtime changes in asset availability or system configuration.

The adoption of the AAS is increasing as it provides a standardized digital representation of industrial assets and improves interoperability. Abdel-Aty et al. [2] describe AAS as an important component for building digital twins by linking physical assets with their cyber representations. AAS stores asset metadata, capabilities, and operational states, enabling consistent access to asset information across the system. Its submodels can also be exported to commonly used industrial formats such as JSON, XML, and AutomationML, allowing integration with existing data infrastructures.

Several studies have investigated AAS applications in manufacturing. Ye et al. [21] demonstrate the use of AAS in a robot-based manufacturing system to support plug-and-produce capabilities. Fan et al. [22] design a flexible manufacturing system using AAS to integrate modular components. Vaage [23] presents a demonstrator enabling real-time communication between factory assets through AAS, while Arm et al. [24] propose automated generation and integration of AAS models to maintain compatibility with legacy equipment. Quadrini et al. [25] further demonstrate how AAS can function as an interoperable software layer in industrial environments.

Building on this foundation, Sakurada et al. [26] propose an agent-based extension of AAS in which digital asset models act as autonomous agents for distributed control. Xia et al. [27] further explore generating AAS instances using LLM agents to support semantic interoperability in Industry 4.0 systems.

Motivated by these developments, the proposed framework combines AAS-based metadata with LLM agents to enable flexible task interpretation and dynamic task allocation. By grounding reasoning in standardized asset metadata, coordination decisions can adapt to changing system conditions without requiring explicit rule-based programming, addressing the limitations of statically engineered coordination logic in conventional industrial communication frameworks.

2.2. Multi-robot systems in manufacturing

Managing complex tasks in modern manufacturing requires a larger number of robots working together efficiently, which can be done with a MRS. MRS is a field of robotics that focuses on using a group of robots, rather than a single robot, to accomplish complex tasks [28]. In such systems, the ability to allocate tasks intelligently is critical. Various methods have been developed to assign tasks effectively among robots, including centralized planning, market-based mechanisms, and optimization algorithms [29,30]. Communication is also a vital component. However, many MRS communication protocols still suffer from issues such as poor interoperability and latency [31].

Due to poor interoperability, current systems face limitations when applied to fully autonomous manufacturing. These systems face challenges in coordinating heterogeneous robots for dynamic task allocation and collision avoidance [32]. Communication protocols remain rigid, system architectures are often vendor-specific, and task allocation mechanisms lack the flexibility to handle runtime changes or unforeseen conditions [29,31]. In autonomous settings, the system must reconfigure itself in real time, adapt to the addition or failure of components, and make decisions independently.

As a result, centralized and optimization-based task allocation methods, although effective under static conditions, rely on predefined models and require manual reconfiguration when system composition changes, limiting their suitability for autonomous manufacturing environments.

2.3. Large language model in robot system

The integration of LLM with robotic systems in recent research has improved autonomous manufacturing environments. LLMs can facilitate human-centric interactions, generalizable collaboration, and seamless execution in Human-Robot Symbiotic Manufacturing ([33]. They enable natural language understanding, task planning, and user interaction, improving robot autonomy and adaptability [34]. LLMs like GPT-4 and PaLM-E are being deployed to enhance robots' comprehension within the industrial automation [35].

Early research has focused on using LLMs to interpret user instructions and generate task plans. Systems such as ProgPrompt and Text2Motion have demonstrated how LLMs can convert natural language into executable robot actions or feasible plans [36,37]. RoboChat and LLM-BRAIn propose unified or symbolic frameworks in which LLMs serve as a central planner, bridging the gap between unstructured input and structured robotic tasks [38,39]. These models enable robots to understand complex instructions, perform object manipulation, and carry out multi-step actions in service and assistance contexts [40,41].

In multi-agent manufacturing systems, LLMs enable agents to communicate in natural language, interpret human instructions, and make decisions, improving adaptability and coordination [42]. Liu et al. show LLM-based frameworks with human input improve robotic performance during manipulation tasks [43]. Other works have explored the interactive use of language in real-time [44], as well as applying LLMs to specialized applications such as feeding assistance [45] and autonomous grasping [46]. These studies indicate LLM integration and developing sophisticated agent communication protocols has been lacking [42].

However, challenges remain, including grounding language to physical actions and real-time processing limitations [35], as well as struggles with retrieving accurate real-world information [47]. These limitations highlight the need for real-time updates in the reasoning source, combined with the reasoning capabilities of LLMs. When robot availability or system configuration changes, prompts or control logic must be manually updated, which limits scalability and interoperability in heterogeneous manufacturing environments.

2.4. Limitations in current approaches and research gaps

Despite significant advancements in Industry 4.0, collaborative robotics, and the integration of LLMs, several limitations persist that hinder the realization of autonomous, interoperable, and scalable manufacturing systems.

- (1) Smart factories still struggle with dynamic adaptability. Conventional systems rely heavily on predefined workflows and rigid task assignments, which limit responsiveness to runtime changes, shifting production demands, or system reconfiguration.
- (2) MRS often operate within vendor-specific architectures, leading to poor interoperability and limited scalability. Existing communication interfaces are not designed to support flexible coordination across heterogeneous robots, and most approaches require substantial reconfiguration when any device or capability changes.
- (3) While LLMs have shown potential in robotic automation, many existing LLM-based frameworks embed the reasoning model as a single centralized component rather than as independent agent processes. As a result, any change to the robot configuration or skill set requires updating the central planner. In addition, these frameworks typically rely on separate perception modules to determine agent status or available skills, rather than using a standardized representation of asset metadata.

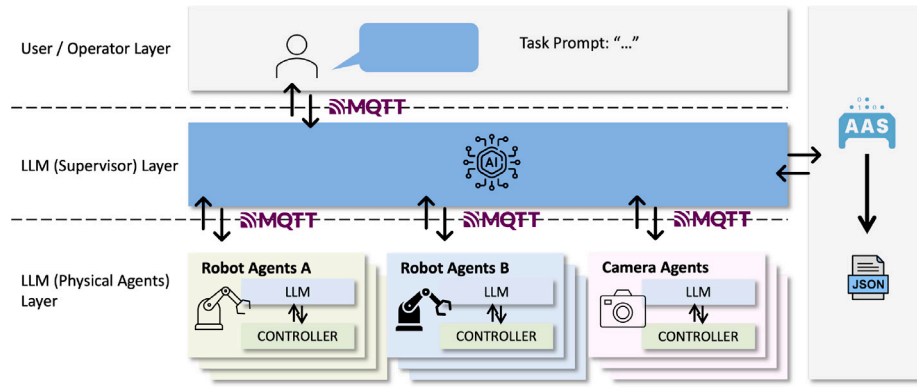


Fig. 1. System architecture of the proposed framework.

Taken together, these gaps point to the need for a scalable framework in which agents can be added or removed without reconfiguring a central controller, an interoperable mechanism that supports collaboration among heterogeneous robots through standardized representations, and a flexible reasoning layer grounded in up-to-date runtime metadata.

In response to these challenges, this paper introduces a unified multi-robot coordination framework that embeds an LLM agent within each physical asset and employs the AAS as a standardized source of operational metadata. The distributed LLM agents interpret user instructions expressed in natural language, retrieve relevant AAS sub-models through RAG, and coordinate task execution without requiring vendor-specific reconfiguration. By representing robot capabilities as human-understandable skills and reflecting each asset's operational state through the AAS, the framework supports plug-and-play integration of heterogeneous robots and enables task allocation to adapt dynamically as system conditions change. This approach eliminates the need for manual reconfiguration when assets change or new capabilities are introduced, allowing coordination to be achieved through language-based interaction.

Taken together, existing approaches lack a unified framework that combines standardized asset representation, decentralized agent-level reasoning, and dynamic task allocation grounded in real-time system metadata. This gap motivates the proposed AAS-driven, LLM-based multi-agent framework for autonomous manufacturing.

3. Framework design

This section describes the framework design of the proposed LLM-driven multi-robot communication system for autonomous manufacturing. The system enables collaborative task execution among heterogeneous agents, such as industrial robots and cameras, coordinated through a centralized Supervisor Agent powered by an LLM. Each physical asset is represented by an AAS that provides structured metadata describing skills and operational states. Using this information, the Supervisor dynamically allocates tasks and delegates responsibilities to agents through MQTT-based communication. The following subsections describe the system architecture, agent pipelines, prompt modeling, and the AAS-based metadata structure supporting coordination.

3.1. System architecture

The proposed system introduces a modular, LLM-driven communication framework for autonomous manufacturing environments. As shown in Fig. 1, the architecture is composed of three main components: the user interface for inputting task prompts, the centralized Supervisor Agent that coordinates the system, and a set of Robot and Camera Agents that execute physical actions.

Each physical asset is represented by AAS, which provides structured metadata about available skills and current status. This metadata is periodically retrieved and used by the Supervisor Agent to construct context-aware task plans through RAG. Instructions are broadcast to all agents via MQTT, while each agent decides locally whether to act based on its identifier and role in the current task step.

This layered design separates high-level reasoning from low-level execution, allowing for scalable coordination, dynamic task adaptation, and plug-and-play extensibility across heterogeneous devices. Because each agent is implemented as a stand-alone module with its own embedded LLM and AAS interface, new robots or cameras can be introduced without modifying the central logic. Additional asset types can be integrated by defining their skills and operational states within the AAS metadata and linking them to local execution functions that interface with the asset controller. This distributed configuration supports plug-and-play scalability, enabling flexible system reconfiguration compared to traditional MRS architectures that require manual code changes when components are replaced. The LLM serves as the reasoning core of the framework, interpreting high-level natural language commands and generating task allocations grounded in AAS metadata. Unlike traditional rule-based planners that rely on predefined decision logic, the LLM dynamically constructs coordination messages based on user instructions and the current system state.

The Supervisor Agent receives user commands in the form of constrained natural language that describes the intended high-level task. A typical input is "Move the box to Station B". The LLM interprets the command to identify the action, object, and destination parameters, and then checks feasibility against AAS metadata that contains each robot's operational state, available skills, and workspace configuration. Based on this information, the Supervisor Agent decomposes the command into subtasks and assigns them to the appropriate Camera and Robot Agents. The Supervisor Agent also includes an exception-handling mechanism that detects incomplete, unsupported, or infeasible user instructions and generates natural-language clarification messages. The specific exception behaviors are demonstrated in the experiment section.

The following subsections describe the internal pipelines of the Supervisor Agent, Robot Agent, and Camera Agent, detailing how each component operates during the collaborative task execution process.

3.1.1. Supervisor Agent pipeline

The Supervisor Agent acts as the central coordinator that interprets user commands and dynamically delegates tasks based on the current system context. Unlike Robot or Camera Agents, which focus on execution, the Supervisor performs planning, task allocation, and synchronization across agents. The overall pipeline of the Supervisor Agent is illustrated in Fig. 2.

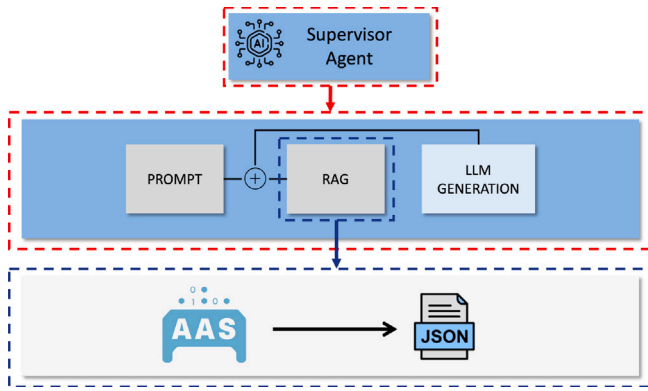


Fig. 2. System pipeline of the Supervisor Agent.

The pipeline begins by constructing a prompt from the user's high-level instruction. Relevant metadata is retrieved from the AAS using a RAG mechanism and incorporated into the prompt context. The retrieved information includes each asset's operational state, available skills, and configuration parameters represented in JSON format. This context enables the LLM to generate structured task delegation instructions that are broadcast to all agents through MQTT.

When multiple agents possess similar skills, the Supervisor evaluates the retrieved metadata to determine the most appropriate executor. Factors such as operational state and workspace feasibility are considered during reasoning. For example, if two robots can perform a *Pick* operation, the Supervisor selects the robot that is currently idle or best positioned to perform the task. In this way, broadcast communication is converted into context-driven task assignment based on real-time system conditions.

In this framework, RAG serves as a metadata retrieval mechanism that supplements LLM reasoning with the latest information from the AAS [27]. Before generating or delegating each instruction, the Supervisor queries the AAS to retrieve relevant metadata entries describing asset capabilities and states. The retrieved data is inserted into the prompt context, allowing the LLM to reason over the current system configuration when producing task allocations.

Fig. 3 illustrates the operational workflow. After receiving a user command, the Supervisor retrieves the latest AAS metadata and generates a task allocation that is broadcast to all agents. As each task step completes, the Supervisor retrieves updated metadata and generates the next instruction accordingly. This iterative process ensures that task decisions always reflect the current system state.

Direct peer-to-peer communication between lower-level agents is not used in the current design. Instead, all coordination is routed through the Supervisor Agent, which validates each task transition using the latest AAS metadata before issuing the next instruction. This supervisory control prevents agents from acting on outdated information and ensures consistent coordination in dynamic manufacturing environments.

3.1.2. Physical agents pipeline

The physical agents in the framework, which are robots and cameras, share the same operational structure. Each physical asset runs an independent LLM agent configured on a separate machine, paired with a local controller and a function list derived from its AAS-defined skills. This design allows both robots and cameras to interpret delegated instructions, select the appropriate skill, and execute the required action without centralized hardcoded logic. The unified pipeline of physical agents is shown in Fig. 4.

As illustrated in Fig. 4, each agent listens for broadcast messages from the Supervisor Agent. Upon receiving a task instruction, the agent checks whether it is responsible for the current step in the

task allocation list. If selected, it parses the instruction, identifies the corresponding function from its AAS-based skill library, and executes the associated function through the local controller. Robots execute motion-level actions such as *PickObject*, *PlaceObject*, or *RotateArm*, while cameras execute perception tasks such as *Capture* or *ObjectDetection*. After completing the action, each agent sends a task-completion reply, along with output data if applicable (e.g., detected coordinates), back to the Supervisor Agent.

Fig. 5 summarizes the behavioral flow shared by both robots and cameras. Agents continuously listen for messages, check responsibility for the current step, execute their AAS-defined skill when selected, and return task results. This loop continues until the Supervisor confirms task completion. Although robots and cameras implement different physical capabilities, their agent logic and message-handling mechanisms are unified under the same framework. This consistent structure enables plug-and-play integration of heterogeneous assets and supports dynamic task reallocation without modifying agent-level configurations.

Fig. 6 illustrates the separation between coordination and execution in the proposed framework. Physical agents execute tasks using local control functions, while collaboration logic is handled by the Supervisor Agent through LLM-based communication grounded in AAS metadata.

3.2. System prompt modeling

In the proposed framework, each agent operates as an independent LLM-driven component that interprets messages, evaluates task requirements, and executes actions according to its assigned role. To ensure consistent reasoning and predictable behavior across agents, a structured system prompt is defined for every agent. This prompt serves as the internal rule base of the LLM, specifying the agent's responsibilities, required inputs, operational constraints, and output format. By modeling the prompt structure explicitly, each agent can interpret AAS metadata, follow coordination rules, and interact reliably with other agents through natural-language messages.

The Supervisor Agent, as depicted in Fig. 7, uses a system prompt that defines how user instructions, AAS metadata, and task rules are combined to produce task allocations and adaptive reallocation decisions. It also includes rules for verifying agent availability, confirming step completion, and handling exceptional cases.

Physical Agents (robots and cameras), as shown in Fig. 8, follow a system prompt that defines how delegated steps are interpreted, how skills encoded in AAS are mapped to local Python functions, and how execution results are communicated back to the Supervisor Agent. The prompt ensures that each agent acts only when assigned and that its behavior is aligned with its AAS-defined capabilities.

3.3. AAS-driven metadata modeling

In the proposed system, asset information is structured using the AAS standard [3]. Each physical component, including robots, cameras, and tools, is represented as an AAS instance composed of multiple submodels describing asset capabilities and operational status. In this work, the AAS for each asset was created using standard AAS modeling tools and stored in JSON format, allowing metadata to be directly retrieved by the LLM without additional conversion.

Each asset entry contains submodels describing attributes such as skills, operational state, identification, and communication endpoints. This modular structure allows assets to be added, removed, or reconfigured without modifying the overall system logic. The AAS therefore serves as the primary knowledge source for the retrieval-augmented generation (RAG) process used by the Supervisor Agent.

Among the available attributes, *Skills* and *OperationalState* are particularly important for task reasoning. *Skills* in the submodel defines

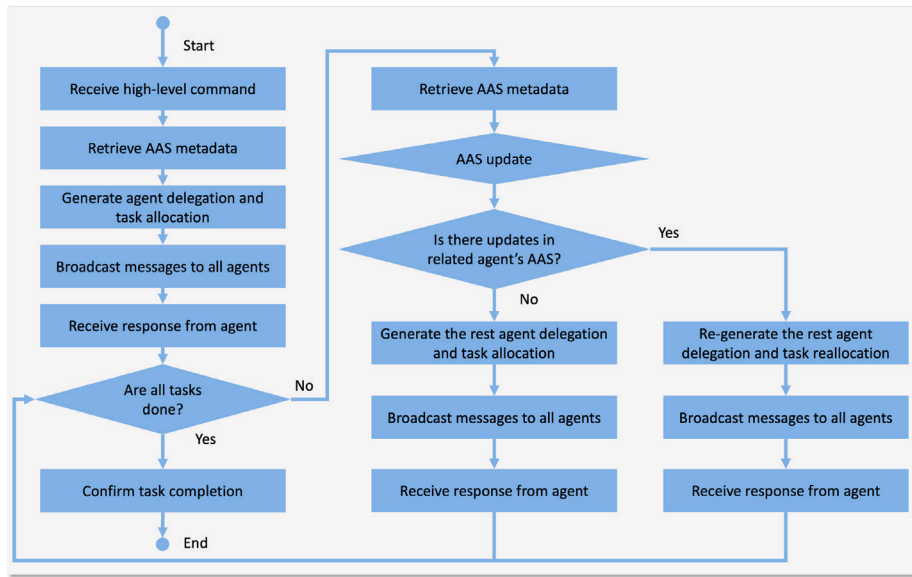


Fig. 3. Operational workflow of the Supervisor Agent.

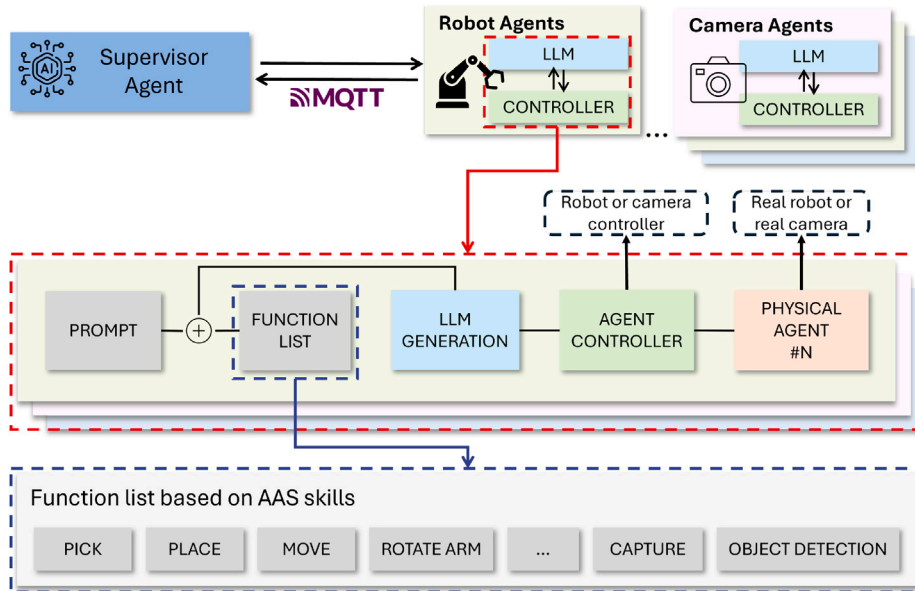


Fig. 4. Unified system pipeline for physical agents (robots and cameras).

the executable capabilities of an asset (e.g., *PickObject*, *Capture*, or *RotateArm*), described using natural language to ensure compatibility with LLM-based reasoning. Each skill corresponds to a predefined execution function within the associated agent configuration. *OperationalState* in the submodel represents the current availability of the asset (e.g., *Idle* or *Busy*), enabling the Supervisor Agent to consider runtime system conditions during task allocation.

By retrieving these metadata elements during task reasoning, the Supervisor Agent can generate task allocations that respect both asset capabilities and current operational states. This mechanism enables adaptive task coordination without relying on hardcoded planning logic. Because all assets expose their capabilities through a standardized AAS schema, the framework supports interoperable coordination across heterogeneous devices from different vendors. The hierarchical structure of the AAS instances used in the experimental testbed is illustrated in Fig. 10.

3.4. Execution constraints and validation mechanisms

LLMs are inherently probabilistic and may generate outputs that are syntactically valid but operationally unsupported, including hallucinated actions or invalid state transitions. In the proposed framework, RAG is used to ground LLM reasoning in up-to-date AAS metadata, ensuring that task decisions reflect the current system state rather than static, prompt-embedded descriptions.

To mitigate invalid actions and unsupported behaviors during execution, the proposed framework incorporates multiple validation and guard mechanisms at different stages of the coordination pipeline. First, all executable actions are constrained to skills explicitly defined in the AAS and mapped to predefined functions within each physical agent. As a result, even if the LLM generates an unsupported or hallucinated action, it cannot be executed unless a corresponding function exists in the agent's skill library. Second, task allocation is conditioned on the operational state retrieved from the AAS. Only agents marked as

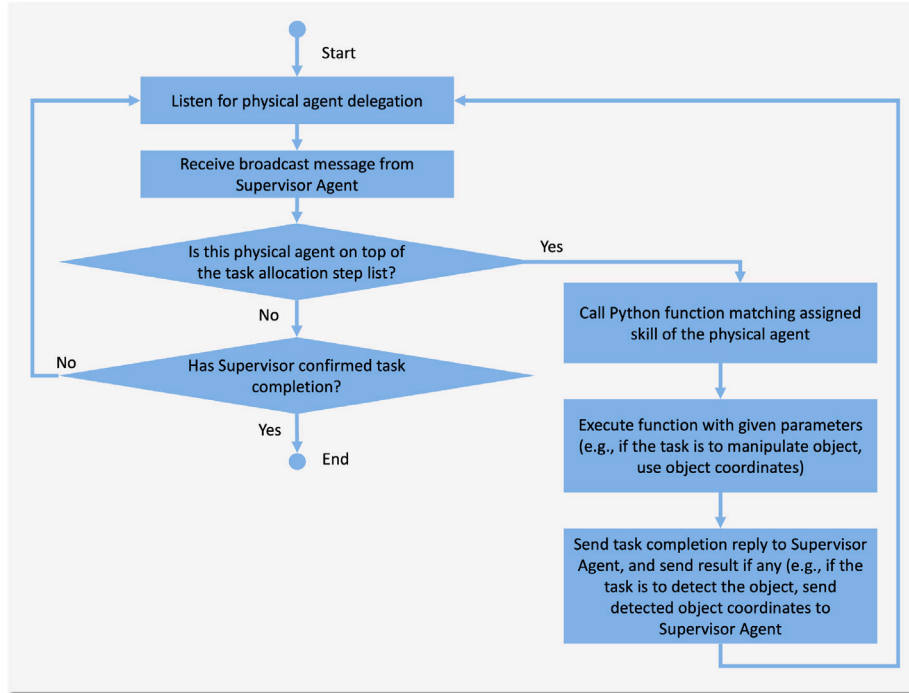


Fig. 5. Operational workflow shared by all physical agents.

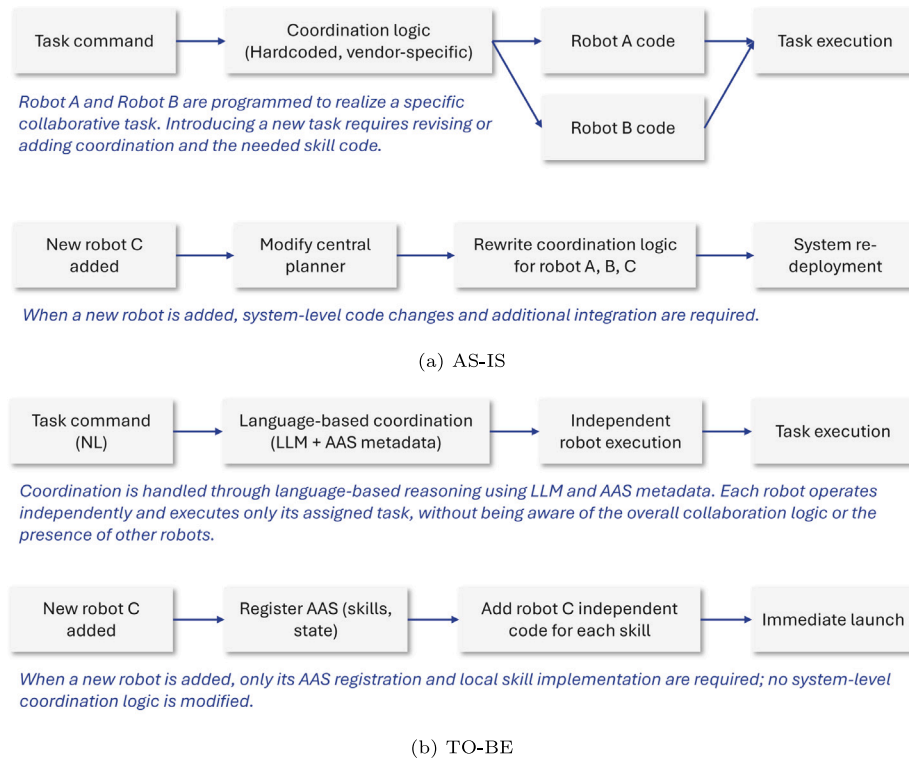


Fig. 6. Comparison of inter-robot coordination workflows. In traditional systems (a), collaboration logic is embedded in robot-specific implementations, requiring system-level code changes when tasks or assets change. In the proposed framework (b), coordination is handled at the communication level using LLMs and AAS metadata, allowing each robot to execute tasks independently without embedding collaboration logic.

System Prompt structure of Supervisor Agent

Representative prompts and key settings of Supervisor Agent, showing how user instructions, AAS metadata, internal rules, and exception-handling mechanisms are used to generate consistent task delegation and adaptive reallocation. Prompt is written in JSON-like format for LLM's better understanding.

```
{
  "role": "Supervisor agent",
  "description": "Decompose user commands into task steps and delegate them to agents based on AAS-defined skills and states. ...",
  "inputs": {
    "user_prompt": "Natural language command, e.g., 'move box to target position'",
    "aas_metadata": {
      "skills": "...",
      "operational_state": "...",
      "relations": "robot-gripper-camera links"
    },
    "agent_reports": "[AgentName] <message> Physical agents"
  },
  "rules": [
    "Delegate only agents with OperationalState in {'Idle', 'Ready'} and required skills.",
    "Use AAS to check attached gripper state for robot assignments.",
    "After each step, output feedback, updated agent delegation, and remaining task allocation.",
    "Do not assume information that is not in AAS or the user prompt.",
    "Always respond using the defined structured response format."
  ],
  "exception_handling": {
    "invalid_skill": "...",
    "missing_parameters": "...",
    "unavailable_agent": "...",
    "unretrieved_aas_update": "...",
    "unexpected_message": "..."
  },
  "supervisor_behavior": {
    "on_error": "Reallocate step based on updated AAS state.",
    "on_failure": "Report error, update allocation, and continue with remaining steps."
  },
  "response_format": {
    "feedback": "confirmation and AAS change if applicable",
    "agent_delegation": "robot <robot_agent>, gripper <gripper_agent>, camera <camera_agent>",
    "task_allocation": [
      "step: <n>, agent: <agent_name>, action: <natural-language action description>"
    ]
  },
  "example": {
    ...
  }
}
```

Fig. 7. System prompt structure of the Supervisor Agent. The prompt outlines how natural-language commands, AAS metadata, and internal coordination rules guide the generation of task sequences and delegation messages. It also includes exception-handling entries that support adaptive reallocation when conditions change.

System Prompt structure of Physical Agent

Representative prompts and key settings of the Physical Agent, showing how delegated task steps, AAS-defined skill mappings, and local execution rules are used to trigger the correct Python functions and report completion. The prompt is written in a JSON-like format for LLM's better understanding.

```
{
  "role": "Physical agent (Robot or Camera)",
  "description": "Execute assigned steps using AAS-defined skills mapped to local Python functions. ...",
  "inputs": {
    "delegated_step": "Step assigned by Supervisor",
    "local_aas_skills": ["..."],
    "task_parameters": "Parameters from Supervisor, e.g., Coordinates"
  },
  "rules": [
    "Act only when this agent is assigned for the current step.",
    "Match delegated action to a local Python function.",
    "Execute with provided parameters.",
    "Send completion message, including detection results if applicable."
  ],
  "response_format": {
    "agent_name": "<this_agent>",
    "status": "completed / failed",
    "message": "optional detection result or confirmation"
  }
}
```

Fig. 8. System prompt structure of the Physical Agent. The prompt specifies how delegated tasks are matched to AAS-defined skills, how local execution functions are triggered, and how completion messages or perception results are returned to the Supervisor Agent.

Idle are considered eligible for task assignment, preventing invalid state transitions such as assigning tasks to unavailable or busy assets. Third, the Supervisor Agent validates each task step using updated AAS metadata before delegation. Direct peer-to-peer execution between physical agents is not permitted, ensuring that every state transition

is mediated and revalidated. Finally, unsupported, incomplete, or infeasible instructions trigger exception-handling behavior rather than execution, resulting in exception messages to the user. Together, these mechanisms constrain the LLM's action space and reduce the risk of hallucinated actions or invalid task execution.

4. Experiment and result analysis

This section presents a sequence of experiments conducted to evaluate the performance of the proposed approach. The experiments are structured across four stages: (1) baseline assessment of task allocation across different LLMs, (2) evaluation of dynamic reallocation in response to agent unavailability, (3) analysis of inter-agent communication in coordinate transmission, and (4) demonstration of real-world deployment on physical robotic systems. Each experiment was carried out under a customized system configuration that aligned with its objective.

In this experiment, the user instruction follows a constrained task structure as explained in Section 3.1.1, to ensure consistent step decomposition and evaluation across all models. Instructions outside this format invoke the Supervisor Agent's exception-handling mechanism, rather than affecting the measured success rate. This allows the experiment to focus strictly on the system's reasoning accuracy under controlled input conditions. Extending the framework to support a broader variety of natural-language task descriptions is planned for future work.

4.1. Comparative evaluation of locally deployed LLMs

This experiment was conducted to identify the most suitable open-source LLM for consistent use across all agent roles within the proposed multi-agent architecture. The evaluation focused on models that can be executed locally without requiring access to commercial cloud services. This choice was motivated by practical deployment considerations in industrial environments, including reducing operational costs, avoiding dependency on continuous internet connectivity, and preserving data privacy by keeping production-related information on-premise. These factors are particularly important for factories and SMEs, where cloud-based solutions may be impractical or undesirable.

The experiment targeted the Supervisor Agent, which performs the system's reasoning tasks. The goal was to assess whether different locally deployed LLMs can reliably generate an initial task allocation under the structured constraints defined in Section 3.

To explore the minimum viable model size for this reasoning task, four instruction-aligned LLMs were selected for evaluation: LLaMA 3 8B Instruct, LLaMA 3.1 8B Instruct with a 128k token context window, LLaMA 3.2 3B Instruct, and LLaMA 3.2 1B Instruct. All of these models are publicly available and support local inference using compatible runtime environments. These models were chosen to cover a representative range of parameter sizes, with the aim of identifying the minimum scale at which an LLM can reliably perform rule-constrained task allocation within the proposed framework.

All models were deployed offline on a single local workstation, both on the CPU and GPU. GPU inference used an NVIDIA GeForce RTX 3090. Each model was prompted with an identical user command and AAS structure (23 submodels), and tasked with generating a complete initial task allocation under strict formatting and reasoning constraints. No fine-tuning was applied. The evaluation recorded two metrics: the success rate, defined as the proportion of generated allocations that exactly matched the defined rules, and the average of total generation time, measured from the prompt submission to the completion of the task allocation output.

This experiment focuses on locally deployed models and examines how the number of parameters influences the reliability of task interpretation and delegation. The evaluation aims to identify the minimum model scale that can consistently satisfy the structured reasoning required in the proposed framework. In this experiment, a trial was counted as successful only if the model generated a response that strictly followed the predefined output format and rule constraints, without producing any extraneous content such as internal reasoning, intermediate steps, or unsolicited explanations.

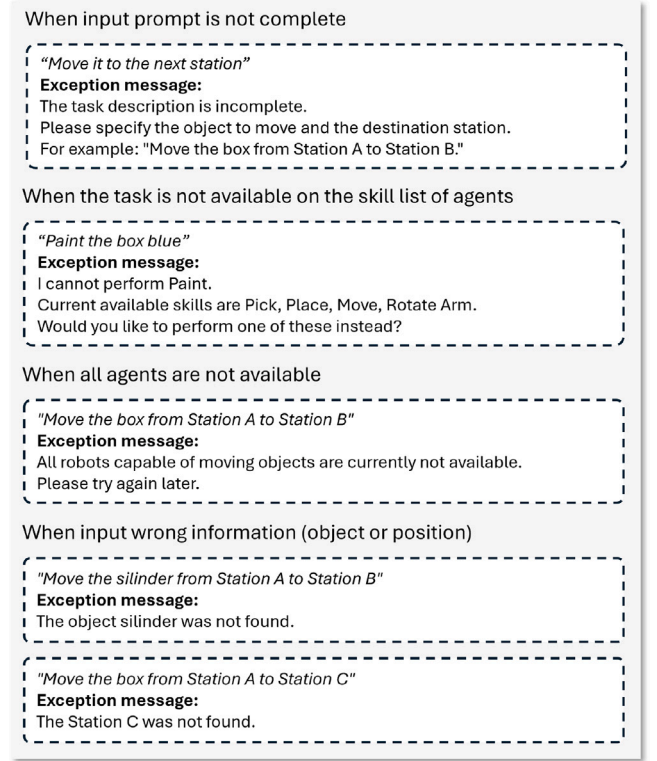


Fig. 9. Exception-handling responses generated by the Supervisor Agent for incomplete, unsupported, unavailable, or incorrect user instructions.

Each model was evaluated over five trials using identical prompts, AAS metadata, and rule constraints. A trial was counted as successful only when the model produced a fully compliant response that matched the predefined output format and rule constraints, without including additional reasoning text or unsupported content. Under these strict criteria, as shown in Table 1, both LLaMA 3 8B Instruct and LLaMA 3.1 8B Instruct 128k generated compliant and fully structured task allocations in all five trials, resulting in a consistent 100% success rate. These success rates were obtained under controlled and deterministic task settings, where task descriptions were well-defined, system states were stable, and no ambiguity or randomness was introduced. This confirms their ability to generate rule-constrained task allocation outputs in the Supervisor Agent context.

Among the two, LLaMA 3.1 8B Instruct 128k demonstrated the shortest average generation time across both CPU and GPU. GPU inference consistently resulted in significantly faster generation times across all evaluated models. For instance, LLaMA 3.1 8B Instruct 128k completed its task in 0.93 s on the GPU, compared to 31.8 s on the CPU, highlighting the performance advantage of GPU acceleration. The 1B and 3B models failed in all five attempts, as they were unable to consistently generate responses that satisfied the defined output constraints. These results indicate the minimum model scale required for reliable rule-constrained reasoning within the proposed framework. Based on these findings, LLaMA 3.1 8B Instruct 128k was selected as the default model for all subsequent experiments. All remaining evaluations in this paper were performed using GPU inference to ensure consistent response time and model performance.

Although the experimental tasks are relatively simple, the framework does not rely on strictly structured instructions. To validate robustness under ambiguous or incomplete user input, we implemented exception-handling behavior in the Supervisor Agent. When a command lacks key information, requests a skill not supported by the available agents, refers to unavailable assets, or contains incorrect

Table 1

Success rate and average generation time of locally deployed LLMs for initial task allocation in the Supervisor Agent. A response is counted as successful only if it strictly follows the predefined output format and rule constraints, without generating extraneous or unstructured content.

Model	CPU		GPU	
	Success rate (%)	Avg. time (s)	Success rate (%)	Avg. time (s)
Llama 3 8B Instruct	100	38.71	100	1.05
Llama 3.1 8B Instruct 128k	100	31.8	100	0.93
Llama 3.2 3B Instruct	0	14	0	0.73
Llama 3.2 1B Instruct	0	10	0	0.6

AAS structure

AAS "Factory_testbed" [https://example.com/ids/aas/7312_1032_6052_7567] of [https://example.com/ids/aas/7312_1032_6052_7567]
Asset "AssetInformation" [https://example.com/ids/asset/1453_1032_6052_0987]
SM "Doosan_01" [https://example.com/ids/sm/1152_1032_6052_1437]
SM "Doosan_02" [https://example.com/ids/sm/4113_1032_6052_8753]
SM "Gripper_2F85_01" [https://example.com/ids/sm/8553_1032_6052_9171]
SM "FANUC_01" [https://example.com/ids/sm/9573_7042_6052_7554]
SM "Gripper_2F85_02" [https://example.com/ids/sm/3183_1032_6052_3904]
SM "Gripper_2F85_03" [https://example.com/ids/sm/4504_1032_6052_8946]
SM "Gripper_2F85_04" [https://example.com/ids/sm/8224_1032_6052_1021]
SM "Gripper_2F85_05" [https://example.com/ids/sm/2054_1032_6052_1272]
SM "Gripper_2F85_06" [https://example.com/ids/sm/3464_1032_6052_5326]
SM "Gripper_Vacuum_01" [https://example.com/ids/sm/6384_1032_6052_1049]
SM "Gripper_Vacuum_02" [https://example.com/ids/sm/0405_1032_6052_5045]
SM "Logi_01" [https://example.com/ids/sm/0355_1032_6052_4104]
SM "Logi_02" [https://example.com/ids/sm/7175_1032_6052_0770]
SM "RealSense_01" [https://example.com/ids/sm/5095_1032_6052_1334]
SM "RealSense_02" [https://example.com/ids/sm/7110_2032_6052_8915]
SM "RealSense_03" [https://example.com/ids/sm/1320_2032_6052_5907]
SM "RealSense_04" [https://example.com/ids/sm/6320_2032_6052_9372]
SM "SCARA_01" [https://example.com/ids/sm/7530_2032_6052_3276]
SM "UR3_01" [https://example.com/ids/sm/9350_2032_6052_5152]
SM "UR3_02" [https://example.com/ids/sm/6481_2032_6052_3700]
SM "UR3_03" [https://example.com/ids/sm/7481_2032_6052_6517]
SM "UR10e_01" [https://example.com/ids/sm/1592_2032_6052_3652]
SM "UR10e_02" [https://example.com/ids/sm/2103_2032_6052_2038]

Robot

SM "UR3_01" [https://example.com/ids/sm/9350_2032_6052_5152]
SMC "Status" (2 elements)
Prop "OperationalState" = Idle
Prop "LastUpdateTime" = 2025-06-16T00:24:41.138967Z
Ref "AttachedGripper" → [Submodel, https://example.com/ids/sm/8553_1032_6052_9171]
SMC "Skills" (2 elements)
Prop "PickObject" = Pick up a specified object from a known location using the attached gripper
Prop "PlaceObject" = Place a held object at a specified target location using the attached gripper
SM "SCARA_01" [https://example.com/ids/sm/7530_2032_6052_3276]
SMC "Skills" (2 elements)
Prop "RotateArm" = Rotates the SCARA arm to align with target object positions
Prop "MoveToPosition" = Moves horizontally in the XY-plane to reach a target position
SMC "Status" (2 elements)
Prop "OperationalState" = Idle
Prop "LastUpdateTime" = 2025-06-16T00:24:41.140245Z

Camera

SM "RealSense_01" [https://example.com/ids/sm/5095_1032_6052_1334]
SMC "Skills" (2 elements)
Prop "Capture" = Capture a still image or frame
Prop "ObjectDetection" = Detect objects and output their coordinates
SMC "Status" (2 elements)
Prop "OperationalState" = Idle
Prop "LastUpdateTime" = 2025-06-16T00:24:41.141391Z

Fig. 10. Asset Administration Shell structure for testbed.

object or station names, the system returns a natural-language clarification message instead of executing an unsafe or undefined action. Fig. 9 presents representative examples of these exception messages, showing how the system manages unstructured input while maintaining safe and interpretable communication.

4.2. Dynamic task reallocation by LLMs in response to AAS updates

This experiment evaluates whether the Supervisor Agent can dynamically reassign tasks when asset states change during execution. Unlike approaches that embed agent status directly in the prompt, the proposed framework retrieves real-time metadata from the AAS through RAG. Each asset (robots, grippers, and cameras) is represented as an AAS instance containing submodels that describe skills and operational state.

Fig. 10 shows the AAS registry used in the testbed. Each asset is represented as an independent entry with submodels describing its capabilities and status. For example, the UR3 robot includes skills for pick-and-place manipulation, the SCARA robot defines motion-related skills such as *RotateArm* and *MoveToPosition*, and the RealSense camera provides vision functions including *Capture* and *ObjectDetection*.

The evaluation focuses on scenarios where the *OperationalState* of an asset changes during task execution. In the current prototype implementation, only this metadata field is updated automatically at runtime. Asset controllers report status changes (e.g., *Idle*, *Busy*, or *Offline*) through an API interface, and these updates are written directly to the AAS JSON representation. Because the Supervisor retrieves AAS

metadata through the RAG mechanism before each reasoning step, the updated state is immediately reflected in subsequent task allocation decisions.

Other metadata elements, such as *Skills* and workspace configuration, are treated as configuration-level information and are updated manually when new capabilities or assets are introduced. These parameters typically remain stable during operation. Therefore, the experiment focuses on runtime updates of *OperationalState*, which act as the primary trigger for dynamic task reallocation.

Three experimental configurations were constructed with different numbers of AAS submodels, as shown in Table 2. These configurations simulate systems of different scales, ranging from five to twenty-three submodels. Each AAS submodel is stored as an individual JSON document, which forms the retrieval corpus used by the RAG module.

In each configuration, the Supervisor first generated an initial task allocation. A failure was then introduced by modifying the *OperationalState* of the assigned robot. The Supervisor was expected to detect the updated AAS metadata and reassign the task to an alternative eligible agent.

Fig. 11 illustrates the task reallocation process. After the Camera Agent completed object detection and returned the object coordinates, the Supervisor retrieved updated AAS metadata before issuing the next instruction. When the originally assigned robot (UR3_02) was marked unavailable, the Supervisor revoked the previous assignment and reallocated the task to UR3_01 with the appropriate gripper. This demonstrates that the system adapts task allocation according to the latest system state.

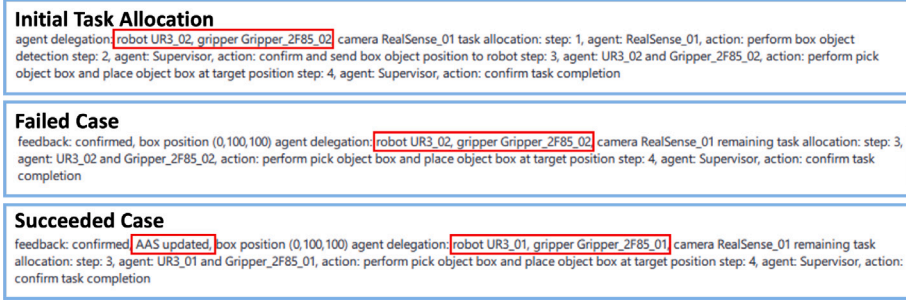


Fig. 11. Dynamic task reallocation based on AAS updates.

Table 2

Evaluation results of dynamic task reallocation under different AAS sizes.

	AAS submodel component				Success rate (%)	Avg. time (s)
	Robot	Camera	Gripper	Total		
Case A	9	6	8	23	100	1.11
Case B	5	4	4	13	100	0.95
Case C	2	1	2	5	100	0.89

Table 3

Comparison between static prompting and RAG-based retrieval under runtime AAS updates.

Method	Avg. time (s)	Success rate (%)	
		Initial AAS	Updated AAS
Static Prompt	1.883	100	0
RAG	1.631	100	100

During development, a short delay was observed between writing updated metadata to the AAS and retrieving it through the RAG interface. To ensure correct synchronization, a brief refresh delay was introduced before each retrieval step. This mechanism allowed the Supervisor to consistently access the latest metadata during reasoning.

Each configuration was evaluated over five trials. As shown in Table 2, the Supervisor Agent achieved a 100% success rate across all configurations, consistently detecting state changes and generating valid reallocated task assignments. Inference time increased slightly as the number of AAS submodels grew, reflecting the additional retrieval overhead introduced by a larger RAG document corpus.

To further evaluate the role of dynamic metadata retrieval, a comparison between static prompting and RAG-based retrieval was conducted under identical task conditions.

As shown in Table 3, both methods succeed when the system state remains unchanged. However, when the assigned robot becomes unavailable during execution, the static-prompt approach fails because the LLM continues reasoning over outdated metadata embedded in the initial prompt. In contrast, the RAG-based approach retrieves the updated AAS metadata and successfully reallocates the task to an available agent. A representative example of the static prompt structure is provided in Appendix A.

4.3. Evaluation of agent communication for data transmission

This experiment evaluates whether the agents in the proposed system can correctly transmit data using prompt-based messaging. Reliable inter-agent communication is essential in the proposed framework, as agents must exchange task parameters to complete collaborative workflows. Among these parameters, object position data (e.g., 3D coordinates) is especially critical, since the execution accuracy of the downstream robot depends on receiving this value without modification.

Table 4

Success rate and average processing time of agents during coordination message exchange tests. The reported time includes prompt construction, LLM inference, MQTT message transmission, and instruction parsing, rather than raw network communication latency. A trial is considered successful only if the transmitted coordinate is preserved exactly across all agents.

Agent	Success rate (%)	Average processing time (s)
Camera Agent	100	5.67
Supervisor Agent	100	4.61
Robot Agent	100	4.69

In this experiment, the Camera Agent acts as the perception endpoint leveraging YOLOv8 model. It reads the detected object position from a CSV file generated by the object detection module and sends this coordinate to the Supervisor Agent as part of its response. The Supervisor must then relay the exact same coordinate to the Robot Agent during task delegation. Finally, the Robot Agent must acknowledge the coordinate it receives, confirming that the numerical value has been preserved throughout the full communication chain. Because each agent runs on a separate machine, this test verifies that no unintended changes occur during distributed message passing. A trial was counted as successful only if the numerical coordinate transmitted by the Camera Agent was received and forwarded by the Supervisor Agent and finally acknowledged by the Robot Agent without any modification or loss.

Fig. 12 shows the result of the data transmission test conducted over five trials without real robot execution. The Camera Agent successfully extracted the detected coordinate (200,100,0) from the CSV file and transmitted it to the Supervisor. The Supervisor then forwarded the same coordinate to the Robot Agent, which confirmed receipt of the exact value. All three agents maintained numerical consistency across messages, demonstrating that the communication pipeline preserves precise task-relevant information.

As shown in Table 4, all agents achieved a 100% success rate, indicating that the transmitted coordinate was preserved exactly across the Camera, Supervisor, and Robot agents in all trials. The Camera Agent exhibited the longest response time due to the additional step of reading the CSV file before generating its message.

The measured timing values represent the average communication and reasoning time required for message processing between agents. These measurements include message transmission and LLM inference but do not include physical execution stages such as object detection or robot motion. The overall inference time in this experiment is higher than in previous tests because the agents were deployed on three different computers communicating through an MQTT broker. This configuration introduces network latency and distributed coordination overhead, reflecting more realistic multi-device deployment conditions.

This experiment verifies that the proposed multi-agent architecture can maintain reliable, lossless, and consistent data transmission in a distributed environment. It confirms that the Supervisor can effectively relay critical information between agents, while Camera and Robot Agents can operate based on accurate and unaltered task parameters.

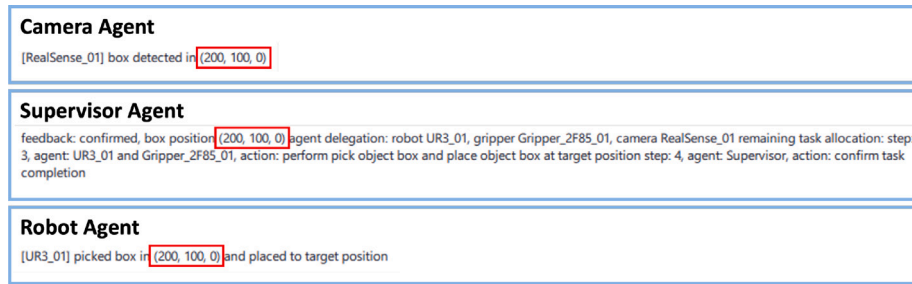


Fig. 12. Prompt-based message passing test for verifying coordinate consistency across Camera, Supervisor, and Robot agents.

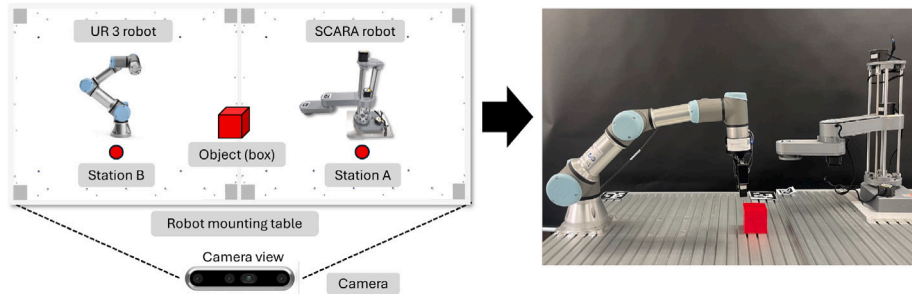


Fig. 13. Physical layout of the experimental setup, showing the location of the agents.

4.4. Real-world execution on physical robotic systems

This experiment was conducted to validate the capability of the proposed system to execute a collaborative task in the real-world deployment using physical robots with different skills, control interfaces, and hardware platforms. The setup includes a SCARA robot, an UR3 robot equipped with a gripper, and an Intel RealSense camera for object detection. Each asset is managed by an individual agent, and each agent communicates through the MQTT-based messaging system. The Supervisor agent is responsible for generating the task allocation prompt based on the provided AAS data and initial command.

Each robot agent is preconfigured with several Python functions that correspond to its available skills. Once the Supervisor agent generates the task delegation prompt, each agent simply invokes the appropriate function with updated task-specific parameters, such as the coordinates of the object.

In this experimental setup, both the SCARA and UR3 robots are provided with their respective reachable workspaces through the initial prompt. The system uses this spatial constraint information to guide task delegation. If the detected object is located within the SCARA's working area, the SCARA is expected to perform the first manipulation step. If the target position is also within its reach, the SCARA completes the full task. However, if the target is outside its range, the SCARA is assigned to move the object to a predefined intermediate point, allowing other robot agent to complete the next task.

Similarly, if the object is initially detected in a position outside the UR3's working area, the UR3 cannot be directly assigned to perform the pick and place operation. Instead, another robot is expected to bring the object to an intermediate point accessible by the UR3. Likewise, if the target position is out of reach for the UR3, it cannot complete the task alone and must rely on coordination with the other agent. This ensures that the task is divided based on real spatial feasibility and highlights the ability of the proposed system to dynamically assign subtasks across heterogeneous robots based on their capabilities and limitations. In the present prototype, the reachable-workspace value serves only as a simplified hint for the Supervisor Agent during task decomposition, as workspace handling is not a central focus of this work. This field is not used for motion validation or any safety-critical decision-making. Fig. 13 illustrates the physical arrangement of the testbed.

The experiment was conducted by initiating an object detection step using the RealSense camera, which successfully located the object and passed the coordinate to the Supervisor agent. The Supervisor then generated a task allocation plan that respected the spatial and skill constraints of the agents, as defined in the AAS.

Fig. 14 illustrates the complete prompt sequence used during execution, showing how the user's natural-language instruction is interpreted by the Supervisor Agent and converted into adaptive, step-by-step delegation messages that demonstrate the advantages of the LLM-based approach compared to traditional rule-based MRS.

In the demonstrated case, the detected object was within the SCARA robot's reachable area, but the target position was outside of its range. As a result, the system correctly assigned the SCARA to move the object to the predefined intermediate point. Although the SCARA lacks a gripper, the corresponding function for moving arm to position was invoked, and the execution was performed as expected, as shown in the upper part of Fig. 15.

Following this, the UR3 robot, which is equipped with a gripper and had the intermediate point within its reach, was tasked with completing the final step by picking the object from the intermediate point and placing it at the target location. This is shown in the lower part of Fig. 15. The execution confirmed that the proposed system can interpret spatial constraints, reflect them through AAS-based reasoning, and coordinate multiple heterogeneous agents in a real-world setting. In this setup, the end-to-end execution time for the full workflow prompt from object detection, delegation, robot motion, and final confirmation, was measured at approximately 53 s. A detailed breakdown of the complete workflow timing from natural-language task input to final task completion is provided in Appendix C.

This result demonstrates the feasibility and robustness of using LLM-based task allocation in combination with AAS and RAG for deploying collaborative multi-agent robotic systems. The execution aligned fully with the predefined scenario and highlights the system's capacity to reason about spatial feasibility, task delegation, and capability matching, even across devices from different vendors.

While the experimental sequence demonstrates a simple pick and place task, the significance lies in the distributed architecture rather than the task complexity. Each agent, including the Supervisor, Camera, and Robot, operates as an independent LLM based process capable of

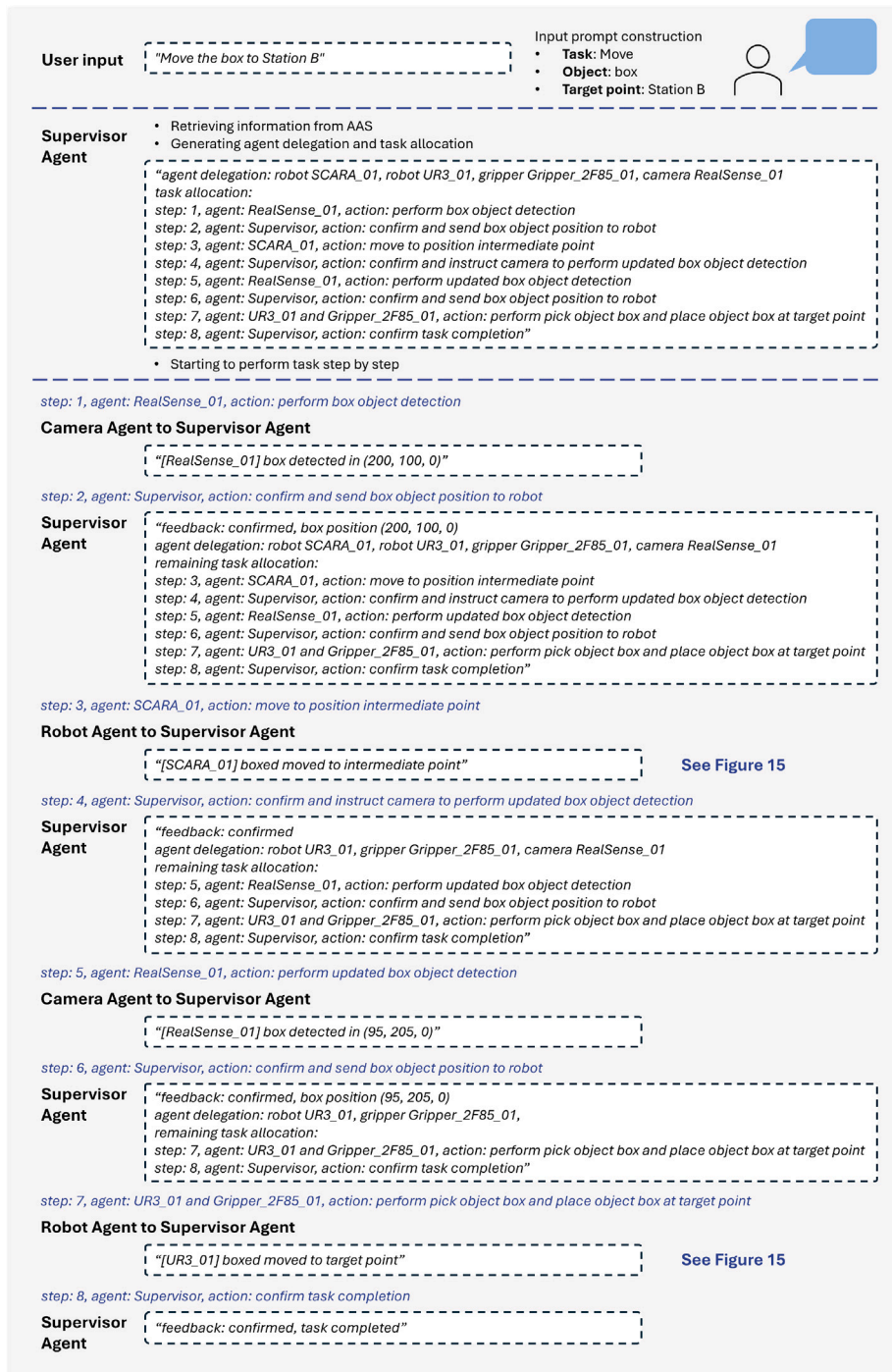


Fig. 14. Prompt implementation workflow of the proposed work.

interpreting and generating communication messages autonomously. This design allows agents to be added, removed, or replaced without the need to reprogram the system logic, showing a degree of modularity and adaptability that is not achievable in conventional rule-based multi-robot systems.

4.5. Local and cloud LLM comparison under different reasoning challenges

This experiment compares locally deployed and cloud-based LLMs under a set of tasks designed to introduce different reasoning challenges. The objective is to evaluate whether the proposed AAS-driven coordination framework operates consistently across models with different reasoning capabilities and deployment environments.

Table 5

Task categories used to evaluate different reasoning challenges in LLM-based task allocation.

Category	Task ID	Primary reasoning challenge
Basic	T1-T2	Direct skill grounding
Moderate	T3-T5	Conditional or ambiguous instructions
Complex	T6-T8	Dynamic state changes/constraint conflicts
Stress	T9-T10	Long-horizon reasoning

The evaluated tasks introduce different reasoning conditions, as summarized in Table 5. These conditions include direct grounding

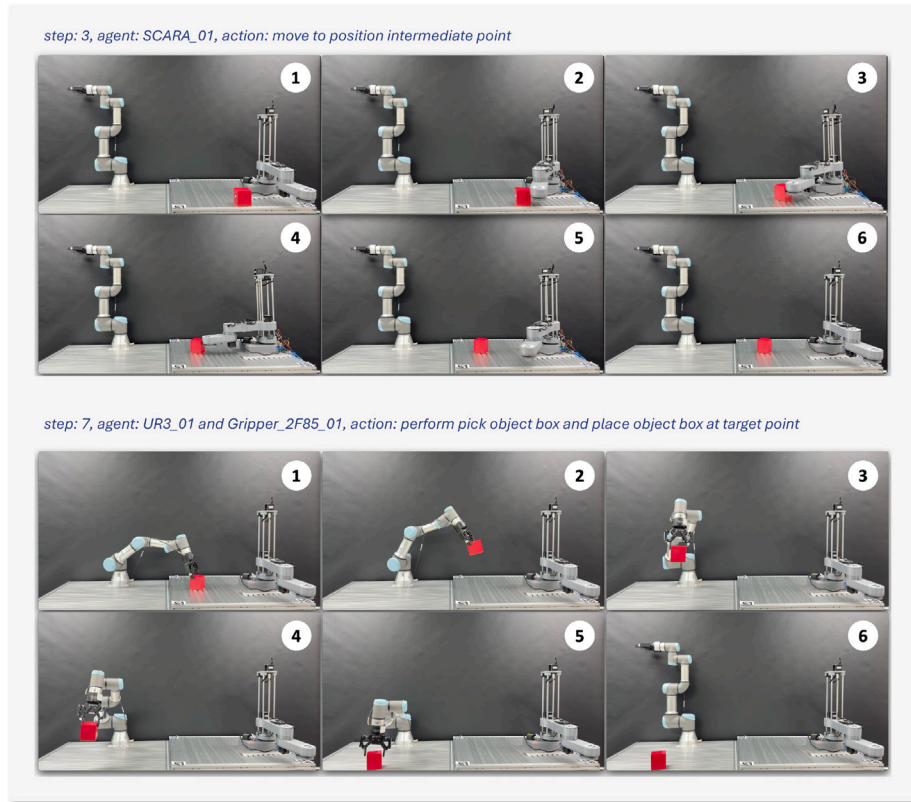


Fig. 15. Real world execution result of SCARA and UR3 robots coordination.

Table 6

Task-level comparison between locally deployed and cloud-based LLMs under identical framework settings. The local model corresponds to LLaMA 3.1 8B Instruct (128k context window) deployed on-premise, while the cloud model corresponds to GPT 5.2 accessed via an online API.

ID	Task description	Local LLM	Cloud LLM
T1	Move the box to Station B	o	o
T2	Pick the box using the vacuum gripper and place it on Station B	o	o
T3	First inspect the box, then place it on Station B	x	o
T4	If the box is detected, move it to Station B; otherwise report an error	o	o
T5	Move the box using any available robot with a gripper to Station B	o	o
T6	Move the box to Station B (update the AAS during runtime)	o	o
T7	Move the box to the correct station (station not specified)	x	o
T8	Move the box to Station B using UR3_01 (while UR3_01 is busy)	x	o
T9	Move the box to Station B and report the final coordinates. If the assigned robot becomes unavailable at any point, select another suitable robot and continue the task.	o	o
T10	Move the box to Station B. After completion, capture an image and report the box's final coordinates. If the assigned robot becomes unavailable at any point, select another eligible robot and continue the task based on updated AAS metadata.	o	o

of instructions to AAS-defined skills (T1-T2), conditional or ambiguous instructions (T3-T5), dynamic system-state updates and constraint conflicts (T6-T8), and longer reasoning chains requiring coordinated multi-step actions (T9-T10). All tasks were executed using the same framework configuration, AAS structure, communication protocol, and prompt constraints. This experiment focuses on the Supervisor Agent, which performs task interpretation, allocation, and reallocation.

Table 6 presents the task-level success comparison between a locally deployed LLM (LLaMA 3.1 8B Instruct 128k) and a cloud-based LLM (GPT 5.2). A trial is considered successful only when the Supervisor Agent produces a valid task allocation that satisfies all framework constraints, including correct agent selection, skill compatibility with AAS-defined capabilities, operational-state validation, and structured output formatting. The tasks were designed as representative reasoning scenarios rather than repeated statistical trials, allowing the evaluation

to highlight specific reasoning challenges relevant to LLM-based coordination. Failures correspond to non-executable or incorrect allocations, such as invoking unsupported actions, selecting incompatible agents, or ignoring asset state constraints.

The locally deployed model performs reliably for tasks where instructions directly correspond to skills defined in the AAS metadata (T1, T2, T4-T6, T9-T10). In these scenarios, the model correctly identifies appropriate robots and generates valid task sequences grounded in the available capabilities. However, failures occur primarily when the instruction introduces semantic ambiguity or constraints that are inconsistent with the available system metadata. For example, T3 includes the instruction "inspect the box", which does not correspond to any defined AAS skill. In this case, the local model attempts to reinterpret the instruction rather than rejecting it, generating a task allocation that assigns a robot to execute the next available manipulation skill even though the requested inspection capability is not defined in the AAS.

Because the selected skill exists within the predefined skill set, it passes the framework-level validation but does not correspond to the intended instruction. Similar reasoning limitations appear in tasks involving underspecified targets (T7) or conflicts between user instructions and asset availability (T8).

In contrast, the cloud-based model successfully handles all evaluated tasks by identifying ambiguous or unsupported instructions and generating appropriate clarification or error responses. Notably, both models succeed in long-horizon tasks (T9-T10), indicating that reasoning difficulty in this context arises primarily from semantic grounding and constraint interpretation rather than the number of execution steps.

These results demonstrate that the proposed AAS-driven coordination framework operates consistently across different deployment settings. Observed differences between the local and cloud models therefore reflect variations in reasoning capability rather than limitations of the framework architecture itself. The results highlight a practical trade-off: locally deployed models provide advantages in privacy, cost control, and on-premise operation, while cloud-based models offer stronger robustness when handling ambiguous instructions or complex constraint reasoning. To further examine whether the framework depends on a specific open-weight LLM family, additional experiments using multiple locally deployed models are presented in [Appendix B](#).

5. Discussion

This work presented a multi-robot communication framework that integrates LLM-based agents with standardized asset metadata to support adaptive task coordination in manufacturing environments. The experimental results demonstrated that the framework can perform task allocation based on current system conditions, dynamically reassign tasks in response to operational changes, transmit task-relevant data between agents, and execute coordinated actions on heterogeneous physical robots. These findings confirm the feasibility of using LLM agents as reasoning components operating over structured industrial metadata rather than as static planners.

A central aspect of the proposed framework is the use of the AAS as a unified representation of each asset's skills and operational state. By converting AAS submodels into a retrievable document corpus, the system enables context-aware prompt construction that reflects runtime conditions. The retrieval-based mechanism allows the Supervisor Agent to reason over updated metadata before each decision step, supporting dynamic reallocation when asset availability changes. The successful deployment of the framework on physical robots from different vendors further demonstrates that standardized metadata can serve as a practical foundation for interoperable coordination in heterogeneous industrial settings.

The comparison between locally deployed and cloud-based LLMs provides additional insight into the practical behavior of the framework under different deployment conditions. The local model performed reliably when task requirements were explicit and directly supported by AAS-defined skills and operational states. However, limitations were observed in tasks involving ambiguous instructions, unsupported actions, or conflicts between user-specified constraints and system availability. In such cases, the local model tended to proceed with implicit interpretations rather than triggering exception handling. In contrast, the cloud-based model consistently identified these situations and returned clarification or error responses. These results indicate that the coordination framework itself remains consistent across deployment settings, while the robustness of ambiguity handling depends on the reasoning capability of the underlying model.

The experimental results also highlight a practical trade-off between deterministic task planners and reasoning-based coordination systems. Traditional rule-based or optimization-based schedulers achieve very low latency because task sequences and communication pathways are

predefined during system engineering. In contrast, the proposed framework introduces additional processing time due to runtime reasoning and inter-agent communication. However, this reasoning capability enables task allocation to adapt dynamically to changes in asset availability or system configuration without requiring manual reprogramming. This flexibility becomes particularly valuable in heterogeneous manufacturing environments where system composition and task requirements may change over time.

The distributed architecture of the framework introduces several deployment considerations. Each physical asset hosts an independent LLM agent, enabling plug-and-play integration and localized execution without centralized hardcoded control. At the same time, this design requires sufficient computational resources at the edge, which may limit deployment in resource-constrained environments. In addition, the current framework relies on a Supervisor Agent for task sequencing and delegation, meaning that failures at this layer can interrupt workflow execution.

Despite these contributions, several limitations of the present work should be noted. The inference latency introduced by LLM reasoning and RAG retrieval may increase as the number of assets and AAS documents grows, and the framework does not guarantee real-time performance. The evaluation focused on structured task descriptions and deterministic physical outcomes, and did not consider uncertainties such as sensor noise, unexpected obstacles, or hardware faults. Furthermore, while execution is constrained by predefined skills and validation mechanisms, the observed failures in handling ambiguous or unsupported instructions highlight the importance of model capability in enforcing these constraints. In addition, while the framework constrains execution to predefined skills defined in the AAS, incorrect semantic interpretation by the LLM may still lead to the selection of a valid but unintended skill when instructions are ambiguous. In the current implementation, such actions would still be executed by the physical agent if triggered by the Supervisor Agent. Therefore, practical deployment in industrial environments requires additional guardrail mechanisms at the execution level. These may include task-level verification, motion validation, collision detection, or safety constraints enforced by the robot controller. Integrating such guardrail mechanisms with LLM-based coordination represents an important direction for future work, particularly for safety-critical manufacturing applications.

Building on these observations, several directions will be explored to further extend the framework. In particular, task-dependent deployment strategies will be investigated to better handle scenarios involving ambiguous instructions or conflicting constraints. This includes exploring adaptive mechanisms that adjust reasoning strategies or backend selection according to task complexity. Further evaluation will also consider larger-scale AAS configurations and less structured task descriptions to assess scalability and generalizability. In addition, incorporating non-deterministic real-world factors and optional human-in-the-loop supervision will be explored to improve reliability in safety-critical manufacturing scenarios.

6. Conclusion

This paper presented a communication framework for autonomous manufacturing in which LLM agents coordinate heterogeneous robotic assets through natural-language messaging grounded in AAS-based metadata. The framework integrates distributed agents, each operating independently on its corresponding physical asset, and a Supervisor Agent that performs task interpretation and delegation using retrieval-augmented reasoning over standardized AAS submodels. This design enables plug-and-play operation, allowing assets to be added, removed, or replaced without reconfiguring system-level logic.

A series of experiments validated the framework's key capabilities, including initial task allocation across multiple LLMs, dynamic reallocation triggered by AAS updates, inter-agent data transmission, and real-world execution with physical robots from different vendors. The

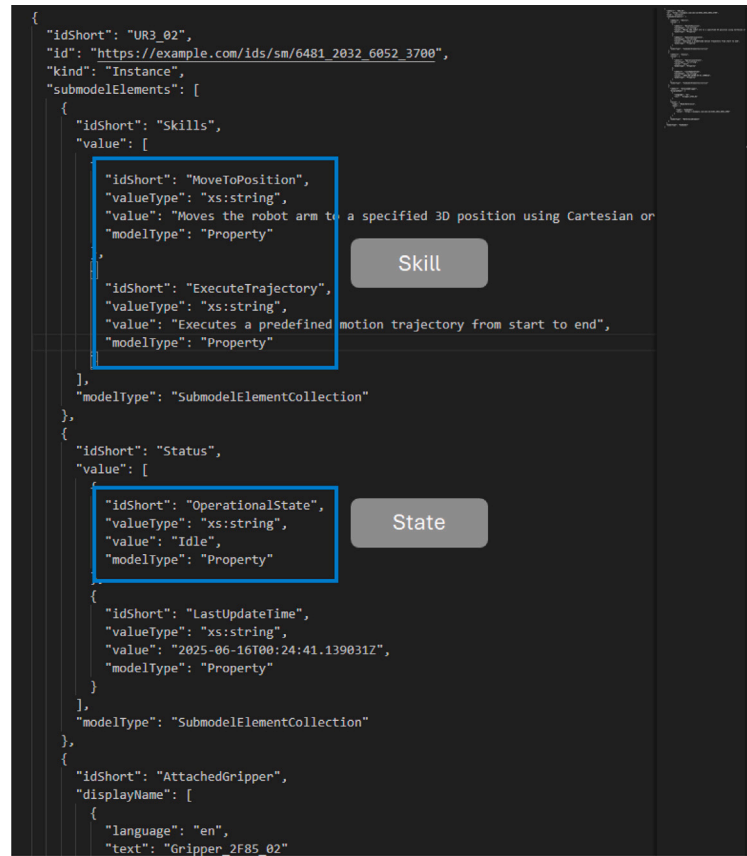


Fig. A.16. Snippet of the AAS metadata embedded in the static prompt. The LLM uses the *Skills* and *OperationalState* fields to determine which robot can execute a task.

results showed that the LLM agents were able to maintain consistent coordination, transmit task-critical parameters without loss, and execute collaborative operations based solely on structured metadata and natural-language messages. The use of RAG ensured that task planning remained sensitive to the most recent operational state of each asset.

These findings indicate that LLM agents can function as reasoning and coordination units in industrial settings when supported by standardized digital representations such as the AAS. At the same time, the comparative evaluation revealed that robustness under ambiguous or conflicting task descriptions depends on the reasoning capability of the underlying LLM, highlighting a practical trade-off between local deployment and cloud-based models. The distributed agent design provides local autonomy while preserving system-level coordination, and the ability to operate offline highlights the potential of this approach in scenarios requiring privacy or localized processing. Overall, the proposed framework offers a scalable and interpretable basis for flexible multi-robot collaboration in modern manufacturing environments.

CRediT authorship contribution statement

Shokhikha Amalana Murdivien: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Jongsu Park:** Writing – original draft, Visualization, Validation, Software, Methodology. **Jumyung Um:** Writing – review & editing, Supervision, Resources, Project administration, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Institute for Information & communications Technology Promotion (IITP) grant funded by the Korea government(MSIP) (RS-2025-02219563, Step by Step Autonomous Manufacturing: From Dataset Construction to Generative AI Utilization), and by Institute of information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. RS-2024-00425383, Graduate School of Metaverse Convergence(Kyung Hee University)).

Appendix A. Static prompt construction and failure analysis

In the static prompting condition, the Supervisor Agent receives the AAS metadata as a JSON representation embedded directly in the prompt. This metadata includes the available assets, their operational states, and their executable skills. An example snippet of the initial metadata used in the experiment is shown in Fig. A.16.

During the experiment, the operational state of one robot was deliberately changed in the AAS to simulate runtime unavailability:

UR3_02 from **Idle** changed to **Busy**

Table B.7

Task-level comparison across three locally deployed open-weight LLM families using the same framework configuration.

Task ID	LLaMA 3.1 8B Instruct 128k	Qwen 2.5 7B	Mistral 7B
T1	o	o	o
T2	o	o	o
T3	x	x	x
T4	o	o	o
T5	o	o	o
T6	o	o	o
T7	x	o	x
T8	x	x	x
T9	o	o	o
T10	o	x	o

However, because the static prompt embeds only the initial metadata and does not retrieve updates during execution, the prompt context still contains the outdated state description. Consequently, the Supervisor Agent may continue allocating the task to UR3_02 even though it has become unavailable. In contrast, the RAG-based approach retrieves the latest AAS metadata before each reasoning step, allowing the Supervisor Agent to detect the state change and reassign the task to an available robot.

Appendix B. Cross-family LLM evaluation

This appendix examines whether the proposed coordination framework depends on a specific open-weight LLM family. Additional experiments were conducted using multiple locally deployed models. In addition to LLaMA 3.1 8B Instruct 128k, two other open-weight models were evaluated: Qwen 2.5 7B and Mistral 7B. All experiments used the same Supervisor Agent configuration, AAS metadata, prompt structure, and task scenarios described in Section 4.5.

The purpose of this experiment is not to benchmark the models themselves, but to verify that the framework can operate consistently across different LLM families without requiring architectural changes. Each model was evaluated using the same ten task scenarios that introduce different reasoning conditions, including direct skill grounding, ambiguous instructions, dynamic state updates, and long-horizon coordination tasks.

Table B.7 summarizes the task-level success results for the three locally deployed models. All evaluated models successfully complete tasks where the user instruction directly corresponds to predefined skills defined in the AAS metadata (e.g., T1, T4-T6, T9-T10). In these cases, the Supervisor Agent correctly interprets the instruction and generates a valid sequence of actions grounded in the available capabilities.

Performance differences appear primarily in tasks that require semantic interpretation beyond explicit skill definitions. For example, T3 includes the instruction “inspect the box”, which does not correspond to any available skill in the AAS metadata. In such cases, some models attempt to reinterpret the instruction by selecting an available manipulation skill rather than rejecting the unsupported request. Similar challenges occur in T7, where the target station is not specified, and in T8, where the instruction conflicts with the current operational state of the robot.

Across the evaluated models, Qwen 2.5 demonstrates the highest robustness in handling these reasoning challenges, successfully resolving most ambiguous or constraint-based scenarios. LLaMA 3.1 performs reliably in tasks grounded in predefined skills but shows occasional failures when instructions introduce unsupported actions or state conflicts. Mistral 7B shows similar behavior, although with slightly lower robustness in scenarios involving ambiguous instructions or strict constraint interpretation.

Despite these differences, all evaluated models successfully execute the majority of tasks that rely on direct grounding in AAS-defined

Table C.8

Estimated end-to-end execution timeline of the demonstrated workflow. Repeated agent-processing steps are distributed around the average values reported in Sections 4.1 and 4.3, while physical execution steps are assigned from the observed task demonstration such that the total matches the measured workflow time of 53.86 s.

Step	Execution stage	Time (s)
1	Supervisor initial task allocation	0.94
2	Object detection	1.28
3	Camera to Supervisor	5.51
4	Supervisor confirmation and delegation to robot	4.68
5	SCARA robot execution	3.80
6	Robot to Supervisor	4.62
7	Supervisor confirmation and delegation to camera	4.47
8	Object detection	1.26
9	Camera to Supervisor	5.03
10	Supervisor confirmation and delegation to robot	4.34
11	UR3 robot execution	12.12
12	Robot to Supervisor	4.26
13	Final Supervisor confirmation	1.55
Total execution time		53.86

capabilities and multi-step task coordination. These results indicate that the proposed framework does not rely on a specific open-weight LLM architecture. Instead, the coordination pipeline, including AAS-based context retrieval, task interpretation, and agent delegation, operates consistently across multiple LLM families. Variations in task success therefore primarily reflect differences in the reasoning capability of the underlying language models rather than limitations of the framework itself.

Appendix C. End-to-end task execution timeline

The full execution timeline of the experimental scenario described in Section 4.4 is presented in this appendix to illustrate the practical timing behavior of the proposed framework. The process begins with a natural-language instruction provided to the Supervisor Agent and concludes with confirmation of task completion after multiple coordination cycles between agents.

The measured execution time for the entire workflow was approximately 53 s. This duration includes task interpretation, object detection, message transmission between agents, robot execution, and confirmation steps. The coordination process involves several reasoning stages where the Supervisor Agent evaluates system metadata and generates updated task instructions before delegating actions to other agents.

Table C.8 summarizes the approximate timing breakdown of the main steps in the workflow.

Data availability

No data was used for the research described in the article.

References

- [1] K. Wei, J. Sun, R. Liu, A review of asset administration shell, in: 2019 IEEE International Conference on Industrial Engineering and Engineering Management, IEEM, IEEE, 2019, pp. 1460–1465.
- [2] T.A. Abdel-Aty, E. Negri, S. Galparoli, Asset administration shell in manufacturing: Applications and relationship with digital twin, IFAC-PapersOnLine 55 (10) (2022) 2533–2538.
- [3] X. Ye, J. Jiang, C. Lee, N. Kim, M. Yu, S.H. Hong, Toward the plug-and-produce capability for industry 4.0: An asset administration shell approach, IEEE Ind. Electron. Mag. 14 (4) (2020) 146–157.
- [4] A. Gupte, R. Wang, V.L. Venkatesh, T. Kim, D. Zhao, Z. Yuan, B.-C. Min, REBEL: Rule-based and experience-enhanced learning with LLMs for initial task allocation in multi-human multi-robot teaming, 2024, arXiv preprint [arXiv:2409.16266](https://arxiv.org/abs/2409.16266).

- [5] K. Arjun, D. Parlevliet, H. Wang, A. Yazdani, Optimizing coalition formation strategies for scalable multi-robot task allocation: A comprehensive survey of methods and mechanisms, *Robotics* (2025).
- [6] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhabri, L. Saldyt, A. Murthy, Lms can't plan, but can help planning in llm-modulo frameworks, 2024, arXiv preprint arXiv:2402.01817.
- [7] M. Javaid, A. Haleem, R.P. Singh, R. Suman, E.S. Gonzalez, Understanding the adoption of industry 4.0 technologies in improving environmental sustainability, *Sustain. Oper. Comput.* 3 (2022) 203–217.
- [8] L. Xu, P.D. Vrieze, H. Yu, K. Phalp, Y. Bai, Interoperability of the future factory: An overview of concepts and research challenges, *Int. J. Mechatronics Manuf. Syst.* 13 (1) (2020) 3–27.
- [9] J. Park, J. Son, S. Cho, J. Um, Smart cognitive HMI with automated knowledge extraction for machine tool, *IEEE Access* (2024).
- [10] A. Tripathy, J. van Deventer, C. Paniagua, J. Delsing, Interoperability between ros and opc ua: A local cloud-based approach, in: 2022 IEEE 5th International Conference on Industrial Cyber-Physical Systems, ICPS, IEEE, 2022, pp. 1–5.
- [11] A. Busboom, Automated generation of OPC UA information models—A review and outlook, *J. Ind. Inf. Integr.* 39 (2024) 100602.
- [12] S. Wang, J. Wan, D. Li, C. Liu, Knowledge reasoning with semantic data for real-time data processing in smart factory, *Sensors* 18 (2) (2018) 471.
- [13] T. Miny, M. Thies, L. Lukic, S. Kabisch, K. Oladipupo, C. Diedrich, T. Kleinert, Overview and comparison of asset information model standards, *IEEE Access* 11 (2023) 99189–99221.
- [14] A. Busboom, Combining OPC UA with semantic web technologies and AI—state-of-the-art and future research directions, *At-Automatisierungstechnik* 73 (7) (2025) 467–481.
- [15] C. Yang, F. Liao, S. Lan, L. Wang, W. Shen, G. Huang, Flexible resource scheduling for software-defined cloud manufacturing with edge computing, *Engineering* 22 (2023) 60–70.
- [16] T. Miny, J. Grothoff, U. Eppe, OPC UA nodestore switch-usage scenarios, *INDIN*, in: 2020 IEEE 18th International Conference on Industrial Informatics, vol. 1, IEEE, 2020, pp. 299–304.
- [17] G. Buchgeher, B. Dorninger, C. Klammer, A. Walchshofer, A. Kern, Migrating cyber-physical systems to OPC UA, *Procedia Comput. Sci.* 200 (2022) 276–283.
- [18] C. Latsou, D. Ariansyah, L. Salome, J.A. Erkoyuncu, J. Sibson, J. Dunville, A unified framework for digital twin development in manufacturing, *Adv. Eng. Inform.* 62 (2024) 102567.
- [19] J.A. Domínguez, R.P. Fuentes, M. Vegetti, L. Roldán, S. Gonnet, M.J. Diván, Ontology implementation of OPC UA and automationml: a review, in: *Advanced Intelligent Technologies for Industry: Proceedings of 2nd International Conference on Advanced Intelligent Technologies (ICAIT 2021)*, Springer, 2022, pp. 17–26.
- [20] F. Mo, J.C. Chaplin, D. Sanderson, G. Martinez-Arellano, S. Ratchev, Semantic models and knowledge graphs as manufacturing system reconfiguration enablers, *Robot. Comput.-Integr. Manuf.* 86 (2024) 102625.
- [21] X. Ye, S.H. Hong, W.S. Song, Y.C. Kim, X. Zhang, An industry 4.0 asset administration shell-enabled digital solution for robot-based manufacturing systems, *IEEE Access* 9 (2021) 154448–154459.
- [22] Z. Fan, D. Shi, O. Meyer, J. Seidelmann, H. Wang, Asset administration shell-based flexible manufacturing system, in: 2023 IEEE 6th International Conference on Industrial Cyber-Physical Systems, ICPS, IEEE, 2023, pp. 1–6.
- [23] A.L. Våge, Industry 4.0 and asset administration shell (AAS): Implementing a small scale demonstrator, *NTNU Open* (2023).
- [24] J. Arm, T. Benesl, P. Marcon, Z. Bradac, T. Schröder, A. Belyaev, T. Werner, V. Braun, P. Kamensky, F. Zzulka, et al., Automated design and integration of asset administration shells in components of industry 4.0, *Sensors* 21 (6) (2021) 2004.
- [25] W. Quadrini, C. Cimino, T.A. Abdel-Aty, L. Fumagalli, D. Rovere, Asset administration shell as an interoperable enabler of industry 4.0 software architectures: a case study, *Procedia Comput. Sci.* 217 (2023) 1794–1802.
- [26] L. Sakurada, P. Leitao, F. De la Prieta, Agent-based asset administration shell approach for digitizing industrial assets, *Ifac-Papersonline* 55 (2) (2022) 193–198.
- [27] Y. Xia, Z. Xiao, N. Jazdi, M. Weyrich, Generation of asset administration shell with large language model agents: Towards semantic interoperability in digital twins in the context of industry 4.0, *IEEE Access* (2024).
- [28] S. Duan, Z. Jiang, Y. Ren, H. Wei, Coordination of multi-robot systems, in: 2022 International Conference on Electronics and Devices, Computational Science, ICEDCS, IEEE, 2022, pp. 361–367.
- [29] H. Chakraa, F. Guérin, E. Leclercq, D. Lefebvre, Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art, *Robot. Auton. Syst.* 168 (2023) 104492.
- [30] R.J. Alitappeh, K. Jeddisaravi, Multi-robot exploration in task allocation problem, *Appl. Intell.* 52 (2) (2022) 2189–2211.
- [31] J. Gielis, A. Shankar, A. Prorok, A critical review of communications in multi-robot systems, *Curr. Robot. Rep.* 3 (4) (2022) 213–225.
- [32] Z. Nie, K.-C. Chen, Hypergraphical real-time multirobot task allocation in a smart factory, *IEEE Trans. Ind. Inform.* 18 (9) (2021) 6047–6056.
- [33] W. Dong, S. Li, P. Zheng, Toward embodied intelligence-enabled human–robot symbiotic manufacturing: A large language model-based perspective, *J. Comput. Inf. Sci. Eng.* 25 (5) (2025).
- [34] Z. Qi, X. Jing, Advances in large language models for robotics, in: 2024 7th International Conference on Mechatronics, Robotics and Automation, ICMRA, IEEE, 2024, pp. 72–76.
- [35] S.H. Aliyu, V.A. Tobiloba, H.T. Adekunle, H.M. Olayinka, K.G. Onuma, A.O. Felix, S. Obafisoye, Integrating large language models in robotics to empower autonomous agents with natural language comprehension capabilities: A comprehensive review, *Int. J. Futur. Eng. Innov.* (2025).
- [36] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, A. Garg, Progprompt: Generating situated robot task plans using large language models, in: 2023 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2023, pp. 11523–11530.
- [37] K. Lin, C. Agia, T. Migimatsu, M. Pavone, J. Bohg, Text2motion: From natural language instructions to feasible plans, *Auton. Robots* 47 (8) (2023) 1345–1365.
- [38] G. Li, X. Han, P. Zhao, P. Hu, L. Nie, X. Zhao, RoboChat: A unified LLM-based interactive framework for robotic systems, in: 2023 5th International Conference on Robotics, Intelligent Control and Artificial Intelligence, RICAI, IEEE, 2023, pp. 466–471.
- [39] A. Lykov, D. Tsetserukou, Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model, in: 2024 2nd International Conference on Foundation and Large Language Models, FLLM, IEEE, 2024, pp. 392–397.
- [40] H. Liu, Y. Zhu, K. Kato, I. Kondo, T. Aoyama, Y. Hasegawa, Llm-based human-robot collaboration framework for manipulation tasks, 2023, arXiv preprint arXiv:2308.14972.
- [41] R. Wang, Z. Yang, Z. Zhao, X. Tong, Z. Hong, K. Qian, LLM-based robot task planning with exceptional handling for general purpose service robots, in: 2024 43rd Chinese Control Conference, CCC, IEEE, 2024, pp. 4439–4444.
- [42] J. Lim, B. Vogel-Heuser, I. Kovalenko, Large language model-enabled multi-agent manufacturing systems, in: 2024 IEEE 20th International Conference on Automation Science and Engineering, CASE, IEEE, 2024, pp. 3940–3946.
- [43] H. Liu, Y. Zhu, K. Kato, A. Tsukahara, I. Kondo, T. Aoyama, Y. Hasegawa, Enhancing the llm-based robot manipulation through human-robot collaboration, *IEEE Robot. Autom. Lett.* (2024).
- [44] C. Lynch, A. Wahid, J. Tompson, T. Ding, J. Betker, R. Baruch, T. Armstrong, P. Florence, Interactive language: Talking to robots in real time, *IEEE Robot. Autom. Lett.* (2023).
- [45] J. Yuan, J. Gupta, A. Padmanabha, Z. Karachiwalla, C. Majidi, H. Admoni, Z. Erickson, Towards an llm-based speech interface for robot-assisted feeding, in: *Adjunct Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, 2024, pp. 1–4.
- [46] M.A. Khan, M. Kenney, J. Painter, D. Kamale, R. Batista-Navarro, A. Ghalamzan-E, Natural language robot programming: NLP integrated with autonomous robotic grasping, 2023, arXiv preprint arXiv:2304.02993.
- [47] J. Yang, H. Jin, R. Tang, X. Han, Q. Feng, H. Jiang, S. Zhong, B. Yin, X. Hu, Harnessing the power of llms in practice: A survey on chatgpt and beyond, *ACM Trans. Knowl. Discov. from Data* 18 (6) (2024) 1–32.